

ARTH.AI

Technical & Theory Deep-Dive

Every formula, every score, every line of logic -
and how each number reaches the screen.

A complete, plain-English walkthrough of the working system
behind the ARTH.AI causal-banking prototype.

Read top to bottom. No prior AI knowledge assumed.

0. How To Read This Document

This guide opens the bonnet of ARTH.AI completely. It is organised as a pipeline: data goes in one end, and a banking action comes out the other. We follow that exact path.

- Plain paragraphs explain the IDEA in everyday words.
- Grey boxes (Courier font) show the ACTUAL formula or rule used in the code.
- Coloured boxes highlight the key takeaway or a real worked number.

Two symbols you will see in formulas:

sigmoid(x)	a squashing function = $1 / (1 + e^{-x})$. It turns any number into a value between 0 and 1. Big input -> near 1, big negative -> near 0.
P(A B)	'the probability of A, given that we observed B'. The bar means 'given'.

Throughout, we trace ONE real customer the system generated - 'Aditya, 36, from Delhi' - so you can see every number change as it moves through the machine.

1. The Big Picture: The Pipeline

ARTH.AI is a chain of 6 stages. Each stage takes the previous stage's output and adds intelligence:

Stage 1 DATA	Generate / read a customer's 180 days of UPI transactions (with consent).
Stage 2 SIGNALS	Turn raw transactions into 12 'signal strengths' (0 to 1) - the clues.
Stage 3 CAUSAL	From the clues, compute the probability of each life event. Pick the most likely.
Stage 4 NEEDS	From that event, rank the banking products it CAUSES (the do-operator).
Stage 5 TIMING	Compute the best day to act, using a counterfactual response curve.
Stage 6 AGENTS	Four agents turn all of the above into concrete actions and messages.

The clever part is Stage 3. Everything else feeds it or acts on it. So we spend the most time there.

2. Theory Primer (the 4 ideas you need)

2.1 Correlation vs Causation

Correlation = two things move together (jewellery buyers often take loans). Causation = one thing makes another happen. ARTH.AI is built to find causes, because acting on a mere correlation leads to wrong, mistimed offers.

2.2 The Structural Causal Model (SCM)

An SCM is a simple map of 'what causes what'. Ours says: a hidden LIFE EVENT is the common cause of BOTH the spending clues AND the banking need. The clues and the need do not cause each other - they are siblings with a shared parent.

LIFE EVENT	(hidden cause)
/	\
spending clues	banking need
(jewellery...)	(loan, account...)

2.3 Bayes' Rule (reasoning backwards)

We can SEE the clues but not the hidden event. Bayes' rule lets us reason backwards: from observed clues, compute how likely each hidden event is. This 'inversion' is the core calculation.

2.4 The do-operator and Counterfactuals

'do(X)' means 'what if we intervene and do X?' - not just observe. We use it to ask: 'if we offer this product NOW vs in 2 weeks, which converts better?' That is a counterfactual (a what-if) and it drives timing.

3. Stage 1 - The Data Layer

Real banks have real UPI data. For a safe, legal prototype we GENERATE realistic fake customers. Crucially, we generate them FROM the causal model, so the data genuinely contains cause-and-effect for the engine to rediscover.

3.1 What a customer looks like

Each customer has: name, age, city, language, estimated income, preferred channel, peak-attention hour, and a list of transactions. Each transaction has a day (0-180, 0=today), an amount (minus=spent, plus=received), and a merchant category (groceries, jewellery, salary, rent...).

3.2 Reproducible randomness (the seed)

We use a 'seeded' random generator (mulberry32). Same seed -> exactly the same customer, every time. This is vital: your live demo will NEVER surprise you, because customer #42 is always identical.

3.3 How a customer is built (step by step)

- Pick a hidden life event by rolling against base rates (priors): e.g. Wedding 9%, Job-change 14%, none 24%.
- Create everyday background transactions: 6 monthly salary credits + 60-100 routine spends (groceries, fuel, dining).
- For the chosen event, fire its signal transactions. Each signal fires with a probability from the EMISSION table. A wedding fires 'jewellery' 84% of the time, 'venue/catering' 82%, etc. Signals cluster in a recent window (~day 35) because clues are LEADING indicators.
- Add 2-11 small payments to other SBI customers (this feeds SPARSH's Trust Radius later).

Key parameters - the EMISSION table (probability a signal fires GIVEN the event):

Wedding: jewellery 0.84, venue/catering 0.82, apparel 0.68

New child: baby/pharma 0.88, hospital 0.62, furniture 0.30

Job change: new-employer salary 0.88, electronics 0.40, apparel 0.35

Relocation: new-city rent 0.86, travel 0.50, furniture 0.45

Medical: hospital 0.92 Education: tuition 0.92, electronics 0.30

Home purchase: large savings inflow 0.90, furniture 0.66

Festival: apparel 0.72, electronics 0.55, jewellery 0.50, travel 0.40

Any signal can also fire by chance at a 3% background rate (noise).

4. Stage 2 - The Signal Engine

This converts messy transactions into 12 clean numbers between 0 and 1 (the 'signal strengths'). For each signal we look at the last 90 days only - recent behaviour matters most.

4.1 How one signal's strength is computed

We count how many transactions in that signal's category appeared recently, and how big they were compared to a 'typical' amount for that signal. Then we squash it to 0-1.

```
avgTicket      = total recent spend / number of recent txns
magnitudeBonus = min(1.2 , avgTicket / referenceAmount)
strength       = sigmoid( 1.35*count + 0.9*magnitudeBonus - 1.7 )
```

'count' (how many times it happened) is the main driver; magnitude refines it. This design means even small but repeated spends (like baby/pharma at Rs 4,000) register as strongly as one big jewellery spend - which is the correct behaviour.

referenceAmount jewellery 50k, venue 90k, baby 4k, tuition 60k, rent 20k, hospital 25k, electronics 35k, large-savings 100k, travel 15k, furniture 25k, apparel 6k, salary 50k (Rs).

4.2 The special one: employer change

A new SALARY merchant name appearing in the recent window (that was not there before) signals a job change. We return 0.9 if a new payroll source is seen, else 0.05. This catches a job change even before other clues appear.

4.3 Network features (the 'graph' view)

Separately, we compute 6 features that describe the customer's transaction NETWORK and rhythm. These feed the agents (especially SPARSH and PRAGATI):

transactionVelocity	recent txn count / older txn count (are they getting more active?).
merchantDiversity	unique categories used / 18 (variety of life).
temporalDiscipline	1 - (spread of salary-credit days)/15 (do they get paid like clockwork?).
trustRadius	number of distinct SBI customers they pay (the social proof signal).
spendingMomentum	recent spend / older spend, capped at 2.5 (acceleration).
incomeStability	0.85 if 4+ salary credits seen, 0.6 if 2+, else 0.4.

Aditya's signals came out: jewellery 0.99, venue/catering 0.85, employer-change 0.90 (a chance firing). Network: diversity 0.44, discipline 1.0, trustRadius 7, momentum 2.5, incomeStability 0.85.

5. Stage 3 - The Causal Inference Engine (the core)

Now the magic. Given the 12 signal strengths, we compute the probability of each of the 9 possible life events, then pick the most likely. This is Bayes' rule applied to our SCM.

5.1 The formula, explained

$P(\text{event} \mid \text{signals})$ is proportional to
 $P(\text{event}) \times \text{PRODUCT over signals of } L(\text{signal}_i)$

where, for each signal i with strength s in $[0,1]$:

$$L = s * P(\text{signal}_i \mid \text{event}) + (1-s) * (1 - P(\text{signal}_i \mid \text{event}))$$

In words: start with how common the event is (the prior). Then, for every signal, multiply in how well that signal's strength matches what this event would produce. A strong jewellery signal strongly supports 'wedding' (which produces jewellery 84% of the time) and weakly supports 'medical' (which almost never does). The strength 's' acts as SOFT evidence - a half-on clue counts half.

5.2 Why we use logarithms

Multiplying many small probabilities makes tiny numbers that computers handle badly. So we add their logarithms instead (adding logs = multiplying values). Then we convert back to clean percentages using the 'softmax' function, which makes all 9 probabilities add up to 100%.

```
logScore(event) = log P(event) + SUM log L(signal_i)
P(event)         = softmax(logScores) -> all events sum to 100%
```

5.3 Explaining WHY (the drivers)

For transparency (and for RBI's explainability rule), we record which signals pushed the winning event up the most, versus pure chance. These become the human-readable 'drivers' shown in the UI.

```
contribution(signal) = log L(signal | winningEvent)
                     - log L(signal | background 3% rate)
```

Keep the top 5 with positive contribution.

5.4 From event to NEEDS (the do-operator)

Once we know the event, we DON'T ask 'what need correlates with jewellery'. We follow the causal arrows from the event to the needs it CAUSES, each with a weight, and rank them.

```
needScore = causalWeight(event -> need) x P(winningEvent)
```

wedding needs	joint account 0.90, personal loan 0.85, gold loan 0.60, credit card 0.45.
daysToNeed	how far ahead the need crystallises: wedding 55, job-change 10, medical 3, new-child 40, home 35 days.

Aditya's result: WEDDING 79.8% (next: job-change 8.2%, none 7.2%). Drivers: jewellery (+3.05), venue (+1.44). Needs: joint account 0.72, personal loan 0.68, gold loan 0.48. Need crystallises in ~55 days.

6. Stage 5 - Counterfactual Timing

Knowing WHAT they need is half the job; knowing WHEN to act is the other half. We model how the chance of conversion changes depending on the day we intervene.

The idea: people convert best a little BEFORE the need fully arrives. Too early = they ignore it; too late = they already solved it elsewhere. So the response curve is a bell shape peaking just before the need.

```
optimalDay = round( daysToNeed * 0.7 )
peakHeight = 0.55 + 0.4 * confidence
response(day) = peakHeight * exp( -(day-optimalDay)^2 / (2*sigma^2) )
sigma is wider before the peak, narrower after (late = punished more)
```

We then read off two points - acting today vs waiting 14 days - and report the difference as the 'timing uplift'.

Aditya (wedding 55 days out): optimal day = 39. Acting TODAY scores 0.38, waiting 14 days scores 0.62. So the engine correctly says: do NOT act today - it is too early; wait closer to day 39. This is the system showing patience, which is exactly right.

7. Proving It Works: Benchmark, Ablation, Robustness

We don't just claim accuracy - we measure it live on fresh fake cohorts. Four models are compared on 'did it recover the right banking need?':

- Causal (full engine): ~70% correct.
- Correlational (pick the single strongest signal, map it straight to a need): ~29%.
- Prior-only (no signals, just guess the most common event's need): ~13%.
- Random: ~12%.

This is an ABLATION: by stripping the engine down piece by piece, we prove each piece earns its place. The jump from 29% (patterns) to 70% (causal) is the value of causal reasoning. ~2.4x better.

7.1 The action-threshold sweep (precision vs coverage)

We only act when confidence clears a bar. Raise the bar -> fewer actions, but each is more likely right. Lower it -> more coverage, less precision. This is a tunable dial.

at threshold 0.6: ~57% of customers acted on, ~81% of those correct
at threshold 0.9: ~13% acted on, ~99% correct

7.2 Robustness (graceful degradation)

We deliberately hide some signals (using a fixed hash so it's repeatable) and watch accuracy fall. It declines smoothly - no sudden collapse - and stays well above random even with 60% of clues hidden. That means the system is robust to missing data.

signals hidden:	0%	15%	30%	45%	60%
need accuracy:	~67%	~59%	~50%	~43%	~39%

7.3 The action threshold in production

We set the bar at 0.6 confidence. Below it, agents stay silent. This is why a 'Stable' customer (no real event) is left alone about 89% of the time - the system refuses to spam on weak evidence.

8. Stage 6 - The Four Agents (every score)

Each agent reads the causal result + network features and produces concrete decisions. Here is exactly how each score is built.

8.1 SPARSH - Acquisition

Decides whether to pursue a prospect, and how.

$$\text{lifeEventScore} = \text{round}((1 - P(\text{none})) * 100 * 0.7 + \text{confidence} * 100 * 0.3)$$

So a customer with a clear, confident life event scores high. Financial-health proxy is read from income stability + payment discipline (Good / Moderate-Good / Moderate / Low). Then the decision tree:

```

if trustRadius >= 5 AND lifeEventScore >= 55 -> ACTIVATE (reach out now)
elif trustRadius >= 3 OR lifeEventScore >= 45 -> NURTURE (slow build)
else -> HOLD (do nothing yet)

```

Aditya lifeEventScore 89, health Good, trustRadius 7 -> ACTIVATE. Sends a WhatsApp account-opening offer in Hindi at 7 PM.

8.2 PRAGATI - Digital Adoption

Computes a Digital Comfort Score (0-100) from 5 behavioural sub-scores, each weighted, then unlocks app features by tier.

```

featureConfidence = merchantDiversity * 1.2
sessionCompletion = 0.4 + temporalDiscipline*0.5
decisiveness      = 0.35 + incomeStability*0.5
lowErrorRate      = 0.5 + (0.3 if velocity>1 else 0.1)
reExploration     = merchantDiversity
score = 100 * (0.28*fc + 0.22*sc + 0.20*dec + 0.18*ler + 0.12*reExp)

```

Tiers 0-30 Beginner (UPI only) | 31-60 Intermediate (FD, goals) | 61-80 Proficient (mutual funds, insurance) | 81-100 Advanced (full suite).

Nudges are sent at the customer's peak-attention hour and never during a detected stress window. The message is built to fit the relevant feature (e.g. an FD nudge computes the exact interest earned).

Aditya score 70 -> Proficient. Sub-scores 0.53/0.90/0.77/0.80/0.44.

8.3 BANDHAN - Engagement

Computes a Financial Empathy Index (FEI, 0-100, higher = more stress) and responds with the right MODE.

```

lateNight = count of small (<Rs2000) debits in last 30 days
recentDebits = count of all debits in last 14 days
FEI = 100 * ( min(1, lateNight/12)*0.4
              + min(1, recentDebits/18)*0.3
              + (0.3 if momentum>1.4 else 0.15 if momentum>1.1 else 0) )

```

Levels 70+ High Stress | 45+ Elevated | 25+ Watchful | else Calm.

If stressed (Elevated/High) -> SUPPORT mode: offers help (0% overdraft, pause FD, free advisor), NOT a sales pitch. If a stable existing customer with a good profile -> TRUST_BUILD mode: proactively offers a lower loan rate (9.2% -> 8.4%). Otherwise -> STEADY (stay quiet).

```
monthlySaving = (income*70) * (9.2 - 8.4) / 100 / 12 [illustrative]
```

Aditya FEI 78 -> High Stress -> SUPPORT. Note: his heavy WEDDING spending (momentum 2.5) reads as financial strain - so BANDHAN gently offers help. An honest, human response.

8.4 GYAAN - Meta-Learning (Financial Twins)

Finds the customer's 'financial twin' cluster and gives peer-validated, privacy-preserved guidance.

```

cohortSize = 60 + round( (1 - P(none)) * 140 )
successRate = 0.55 + confidence * 0.35

```

Aditya cluster ~190 people, success rate 0.83, top action: Joint Savings Account. 'People like you who acted reached their goal 83% of the time.'

9. How The Numbers Reach The Screen

All the maths above lives in plain functions (in the /src/lib folder). The website (React) calls these functions and paints the results. Here is the flow.

9.1 The data flow

```
user picks a customer -> generateCustomer(seed)
-> extractSignals() -> inferLifeEvent() -> timingCounterfactual()
-> runSparsh/Pragati/Bandhan/Gyaan()
-> React renders bars, charts, badges, messages
```

The page uses 'useMemo': it only re-runs the maths when you change the customer, so it is instant and smooth. Nothing is pre-recorded - every bar you see was just computed live in your browser.

9.2 How a 'score bar' is drawn

A score like 0.79 becomes a coloured bar. The MeterBar component takes the value, turns it into a percentage width, and animates from 0 to that width. Posterior percentages, comfort sub-scores, signal strengths - all use this same bar.

```
widthPercent = clamp(0..100, value/max * 100) then animate to it
```

9.3 The charts

The timing curve and the ablation graphs use a chart library (Recharts). We hand it the list of (day, probability) points from timingCounterfactual(), and it draws the smooth area/line. The 'optimal day' dashed line is a marker we add.

9.4 The 5-step demo and the gate

The Live Demo page shows Stages 2-6 as 5 numbered steps. After Stage 3 it checks isActionable():

```
isActionable = (topEvent != none) AND (confidence >= 0.6)
```

If true, it shows steps 3-5 (needs, timing, action). If false, it replaces them with a calm 'hold back - steady customer' panel. That is the spam-prevention behaviour, visible on screen.

9.5 The auto-cycling hero

The home page card cycles through 4 pre-chosen customers every ~4 seconds (a timer in the component), each time re-running the real inference - so even the landing animation is genuine.

10. The Formal Proof (Python + DoWhy)

To convince technical judges beyond our own engine, the /research folder uses DoWhy (Microsoft's real causal-inference library). It runs a clean experiment:

- Build data where, by construction, jewellery has ZERO direct effect on the loan need - the wedding causes both.
- Measure the naive correlation: it wrongly shows a strong +0.47 jewellery->need link.
- Apply back-door adjustment (control for the wedding): the link collapses to ~0.01 - the truth.
- Run refutation tests: a placebo treatment (effect should vanish - it does, p=0.86) and a random common cause (effect should stay - it does, p=0.92).

Conclusion: 97% of what a correlational model 'learns' here is an illusion caused by ignoring the hidden cause. ARTH.AI's approach removes that illusion. The four DoWhy steps are: Model -> Identify -> Estimate -> Refute.

11. Architecture & Compliance Mapping

11.1 Six layers <-> the code

L0 Data	synthetic.ts (consented data simulation).
L1 Signals	signals.ts (extractSignals, networkFeatures).
L2 Causal	causal.ts + scm-knowledge.ts (the SCM brain).
L3 Agents	agents.ts (SPARSH/PRAGATI/BANDHAN/GYAAN).
L4 Action	the messages each agent emits; UI delivery.
L5 Learning	federated-learning blueprint (production roadmap).

11.2 Compliance is built into the logic

- Explainability (RBI FREE-AI): the 'drivers' list (5.3) gives a plain reason for every decision.
- Fairness: decisions follow the causal event, not proxies for caste/religion/gender, structurally avoiding bias.
- Human-in-the-loop: credit actions are framed as recommendations needing approval.
- Privacy (DPDP): per-signal consent; federated learning keeps raw data on-device.

12. Full Worked Example: 'Aditya'

Putting it all together, here is every number for one real generated customer, end to end:

1. Customer	Aditya, 36, Delhi, Hindi, income Rs 85,000, 90 transactions, existing customer, peak hour 7 PM.
2. Signals	jewellery 0.99, venue/catering 0.85, employer-change 0.90 (a chance firing).
3. Causal	WEDDING 79.8% (job-change 8.2%, none 7.2%, festival 4.1%). Drivers: jewellery +3.05, venue +1.44.
4. Needs	joint account 0.72, personal loan 0.68, gold loan 0.48, credit card 0.36. Need in ~55 days.
5. Timing	optimal day 39; act-now 0.38 vs wait-14 0.62 -> be patient, act near day 39.
6a SPARSH	score 89, health Good, trust 7 -> ACTIVATE (WhatsApp, Hindi, 7 PM).
6b PRAGATI	comfort 70 -> Proficient; mutual funds & insurance unlocked.
6c BANDHAN	FEI 78 -> High Stress (wedding spend looks like strain) -> SUPPORT, offers help.
6d GYAAN	~190 twins, 83% success, suggests Joint Savings Account.

Notice the system is not naive: a spurious salary signal fired, yet WEDDING still won; and timing told us to WAIT rather than pounce. That nuance is what separates causal reasoning from pattern-matching.

13. Tech Stack, File Map & Honest Limits

13.1 Files (where each thing lives)

<code>src/lib/types.ts</code>	all data shapes.
<code>scm-knowledge.ts</code>	priors, emission table, needs, timing - the SCM parameters.
<code>synthetic.ts</code>	customer & transaction generator.
<code>signals.ts</code>	signal strengths + network features.
<code>causal.ts</code>	inference, do-operator, timing, benchmark, ablations, gating.
<code>agents.ts</code>	the four agents.
<code>src/app/*</code>	the website pages; <code>src/components/*</code> the UI pieces.
<code>research/*</code>	the Python DoWhy proof.

13.2 Tech

Website: Next.js 14 + TypeScript + Tailwind + Framer Motion + Recharts. Engine: pure TypeScript, runs in the browser (no server, no delay). Proof: Python + DoWhy + EconML. Production blueprint adds LangGraph, PyTorch Geometric (GNN), Neo4j, FastAPI, Flower (federated learning).

13.3 Honest limitations

- All numbers are from synthetic data where we know the truth; real-world accuracy will be lower and must be A/B validated on consented data.
- The browser engine is a faithful but lightweight SCM; production uses DoWhy/EconML and a real graph neural network.
- Account Aggregator data needs explicit consent and cannot be pulled for non-customers; SPARSH uses only consented or opt-in signals.
- Some features (voice, live WhatsApp/DigiLocker, federated training) are blueprint, presented honestly as roadmap.

That is the entire machine - from a raw transaction to a timed, explainable, compliant banking action.